

DAY 13

NAME: Nandhini V

DATE: 21-08-2025

Task:

Create a fresh api, to add details like state, chief minister and language only by admin and view those details only by officer.

Code:

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "Jwt": {
    "Issuer": "https://www.payoda.com",
    "Audience": "students_CSharp",
    "Key": "ThisIsASuperSecretKeyForJwt123456!",
    "ExpireMinutes": 30
  },
  "sql_conn": {
    "conn_string": "Data Source=PTSQLTESTDB01;Initial Catalog=Nandhini_Demo;Integrated Security=true;TrustServerCertificate=True;"
  },
  "AllowedHosts": "*"
}
```

JWTOptions.cs

```
namespace Day13TaskApi.Security
{
    public class JWTOptions
    {
        public string Issuer { get; set; } = "";
    }
}
```

```

        public string Audience { get; set; } = "";
        public string Key { get; set; } = "";
        public int ExpireMinutes { get; set; } = 60;
    }
}

```

JwtService.cs

```

using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace Day13TaskApi.Security
{
    public class JwtService
    {
        public static string CreateJWTToken(JWTOptions options, IEnumerable<Claim> claims)
        {
            var creds = new SigningCredentials(new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(options.Key)),
SecurityAlgorithms.HmacSha256);
            var token = new JwtSecurityToken(
                issuer: options.Issuer,
                audience: options.Audience,
                claims: claims,
                expires: DateTime.Now.AddMinutes(30),
                signingCredentials: creds
            );

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}

```

Program.cs

```

using Day13TaskApi.Context;
using Day13TaskApi.Security;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;

```

```

using System;
using System.Security.Claims;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var sql_section = builder.Configuration.GetSection("sql_conn");
string connectionString = sql_section["conn_string"];

builder.Services.Configure<JWTOptions>(builder.Configuration.GetSection("Jwt"));
var jwt = builder.Configuration.GetSection("Jwt");
var issuer = jwt["Issuer"];
var audience = jwt["Audience"];
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt["Key"]!));
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
}).AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidIssuer = issuer,
        ValidAudience = audience,
        IssuerSigningKey = key,
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateIssuerSigningKey = true,
        ValidateLifetime = true,
        ClockSkew = TimeSpan.Zero,
        RoleClaimType = ClaimTypes.Role,
        NameClaimType = ClaimTypes.NameIdentifier
    };
});

builder.Services.AddDbContext<AppDbContext>(x => x.UseSqlServer(connectionString));

var app = builder.Build();

```

```
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthorization();

app.MapControllers();

app.Run();
```

Election.cs

```
using System.ComponentModel.DataAnnotations;

namespace Day13TaskApi.Models
{
    public class Election
    {
        [Key]
        public int Id { get; set; }

        [StringLength(20, MinimumLength = 3)]
        public string State { get; set; }

        [StringLength(30, MinimumLength = 5)]
        public string ChiefMinister { get; set; }
        public string? Language { get; set; }
    }
}
```

User.cs

```
using System.ComponentModel.DataAnnotations;

namespace Day13TaskApi.Models
{
    public class User
    {
        public int Id { get; set; }

        [StringLength(25, MinimumLength = 5)]
        public string Name { get; set; }
    }
}
```

```

        [StringLength(25, MinimumLength = 5)]
        public string? Email { get; set; }
        public string Password { get; set; }

        [StringLength(25, MinimumLength = 5)]
        public string? Role { get; set; } = "Officer";
    }
}

```

AppDbContext.cs

```

using Day13TaskApi.Models;
using Microsoft.EntityFrameworkCore;

namespace Day13TaskApi.Context
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        public DbSet<Election> Elections { get; set; }

        public DbSet<User> NewUsers { get; set; }
    }
}

```

ElectionController.cs

```

using Day13TaskApi.Context;
using Day13TaskApi.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace Day13TaskApi.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class ElectionController : Controller
    {
        AppDbContext _appDbContext;
    }
}

```

```

public ElectionController(AppDbContext appDbContext)
{
    _appDbContext = appDbContext;
}

[Authorize(Roles = "Admin")]
[HttpPost("AddCM")]
public IActionResult AddCM(Election elec)
{
    _appDbContext.Elections.Add(elec);
    _appDbContext.SaveChanges();
    return Ok("Added state and cm details");
}

[Authorize(Roles = "Officer")]
[HttpGet("GetDetails")]
public IActionResult GetDetails()
{
    var all_details = _appDbContext.Elections.ToList();
    return Ok(all_details);
}
}
}

```

UserController.cs

```

using Day13TaskApi.Context;
using Day13TaskApi.Models;
using Day13TaskApi.Security;
using Microsoft.AspNetCore.Identity.Data;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;

namespace Day13TaskApi.Controllers
{
    public record LoginRequest(string name, string Password);

    [ApiController]
    [Route("[controller]")]
    public class UserController : Controller

```

```

{
    AppDbContext _appDbContext;
    Microsoft.Extensions.Options.IOptions<JWTOptions> jwtOptions;

    public UserController(AppDbContext appDbContext,
Microsoft.Extensions.Options.IOptions<JWTOptions> _jwtOptions)
    {
        _appDbContext = appDbContext;
        jwtOptions = _jwtOptions;
    }

    const string ADMIN = "admin";
    const string OFFICER = "officer";
    [HttpPost("Login")]
    public IActionResult Login(LoginRequest loginReq)
    {
        var name = loginReq.name;
        var hashPassword = GetPasshash(loginReq.Password);
        var logged_in_user = _appDbContext.NewUsers.Where(u => u.Name == name &&
u.Password == hashPassword).FirstOrDefault();
        if (logged_in_user == null)
            return Unauthorized("Sorry, not authorized");
        else
        {
            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Name, loginReq.name),
            };
            if (logged_in_user.Role.Trim().ToLower().Contains(ADMIN))
            {
                claims.Add(new Claim(ClaimTypes.Role, "Admin"));
            }
            if (logged_in_user.Role.Trim().ToLower().Contains(OFFICER))
            {
                claims.Add(new Claim(ClaimTypes.Role, "Officer"));
            }
            var token = JwtService.CreateJWTToken(jwtOptions.Value, claims);
            return Ok(token);
        }
    }
}

[HttpGet("GetUsers")]
public IActionResult GetUsers()

```

```

    {
        var all_users = _appDbContext.NewUsers.Select(u => new { u.Name, u.Email }).ToList();
        return Ok(all_users);
    }

    [HttpPost("AddUser")]
    public IActionResult AddUser(User user)
    {
        User user_with_hashed_pass = new User { Name = user.Name, Email = user.Email,
        Password = GetPasshash(user.Password), Role = user.Role };
        _appDbContext.NewUsers.Add(user_with_hashed_pass);
        _appDbContext.SaveChanges();
        //return Ok("User has been added");
        return Redirect("GetUsers");
    }

    private string GetPasshash(string raw_pass)
    {
        using (SHA256 sha256Hash = SHA256.Create())
        {
            byte[] pass_converted_to_bytes = Encoding.UTF8.GetBytes(raw_pass);
            byte[] bytes = sha256Hash.ComputeHash(pass_converted_to_bytes);

            StringBuilder builder = new StringBuilder();

            for (int i = 0; i < bytes.Length; i++)
            {
                builder.Append(bytes[i].ToString("x2"));
            }
            return builder.ToString();
        }
    }
}

```

Output:

Values in Database - NewUsers & Elections

Results		Messages			
	Id	Name	Email	Password	Role
1	1	Nandhini	nandhu123@gmail.com	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd6...	Admin
2	3	Kiran	kiranrs67@gmail.com	d74ff0ee8da3b9806b18c877dbf29bbde50b5bd8e4dad7a3a...	Officer

Results		Messages		
	Id	State	ChiefMinister	Language
1	1	Kerala	Pinarayi Vijayan	Malayalam
2	2	Tamil Nadu	M. K. Stalin	Tamil
3	3	Karnataka	Siddaramaiah	Kannada
4	4	Punjab	Bhagwant Mann	Punjabi
5	5	Gujarat	Bhupendra Patel	Gujarati

Generating token for user - Admin

Request body

```
{
  "name": "Nandhini",
  "password": "password"
}
```

Server response

Code	Details
------	---------

200

Response body

[illegible]

Download

Adding values by admin user token - Ok

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5278/Election/AddCM
- Body Type:** JSON
- Body Content:**

```
1 {  
2   "State": "Delhi",  
3   "ChiefMinister": "Arvind Kejriwal",  
4   "Language": "Hindi"  
5 }
```
- Response:** 1 Added state and cm details

Getting values by admin user token - Forbidden

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:5278/Election/GetDetails
- Body Type:** none
- Response:** This request does not have a body

Below the main interface, the response details are shown:

- Status:** 403 Forbidden
- Time:** 10 ms
- Size:** 99 B
- Save Response:** (button)

After adding values - Database

Results		Messages		
	ID	State	ChiefMinister	Language
1	1	Kerala	Pinarayi Vijayan	Malayalam
2	2	Tamil Nadu	M. K. Stalin	Tamil
3	3	Karnataka	Siddaramaiah	Kannada
4	4	Punjab	Bhagwant Mann	Punjabi
5	5	Gujarat	Bhupendra Patel	Gujarati
6	6	Delhi	Arvind Kejriwal	Hindi

Generating token for user - Officer

Request body

```
{
  "name": "Kiran",
  "password": "pass"
}
```

Server response

Code

Details

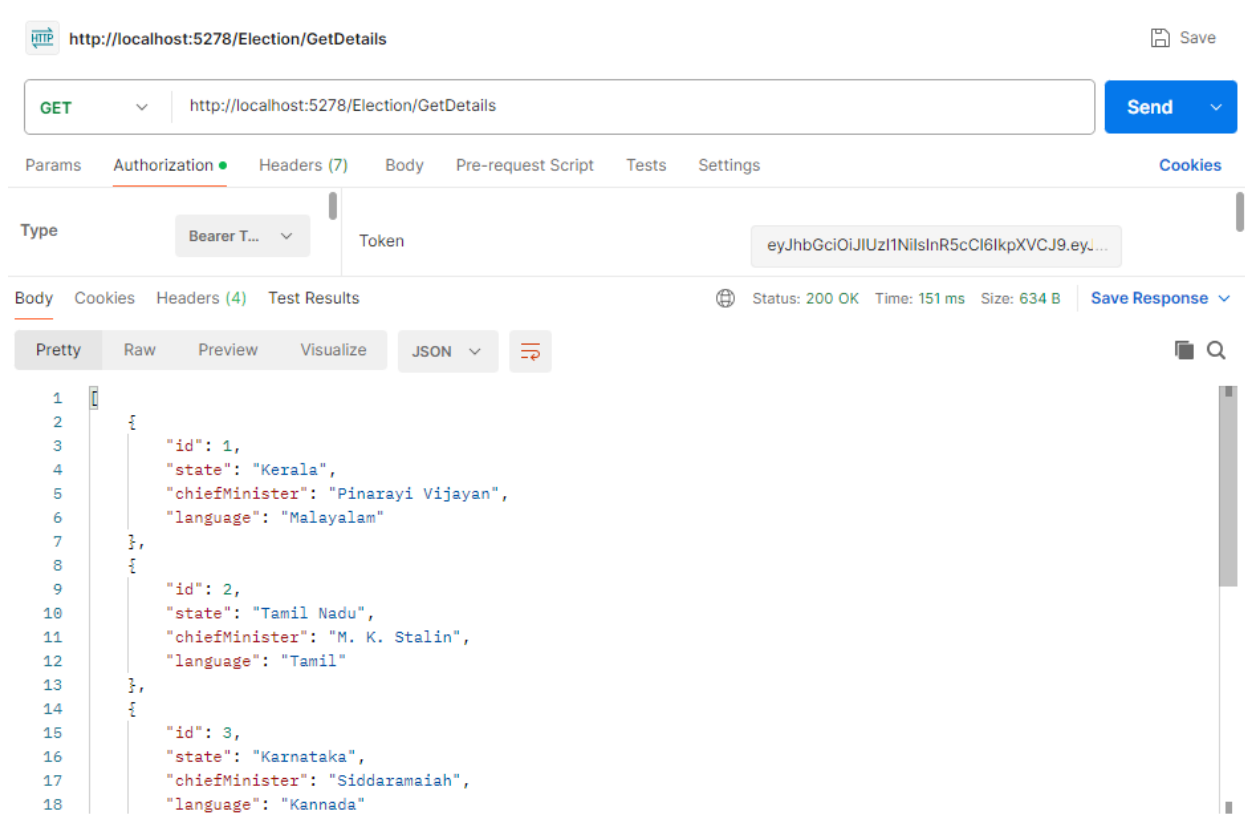
200

Response body

[illegible]

Download

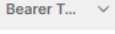
Getting values by officer user token - Ok



HTTP  http://localhost:5278/Election/GetDetails  Save

GET  http://localhost:5278/Election/GetDetails 

Params Authorization  Headers (7) Body Pre-request Script Tests Settings Cookies

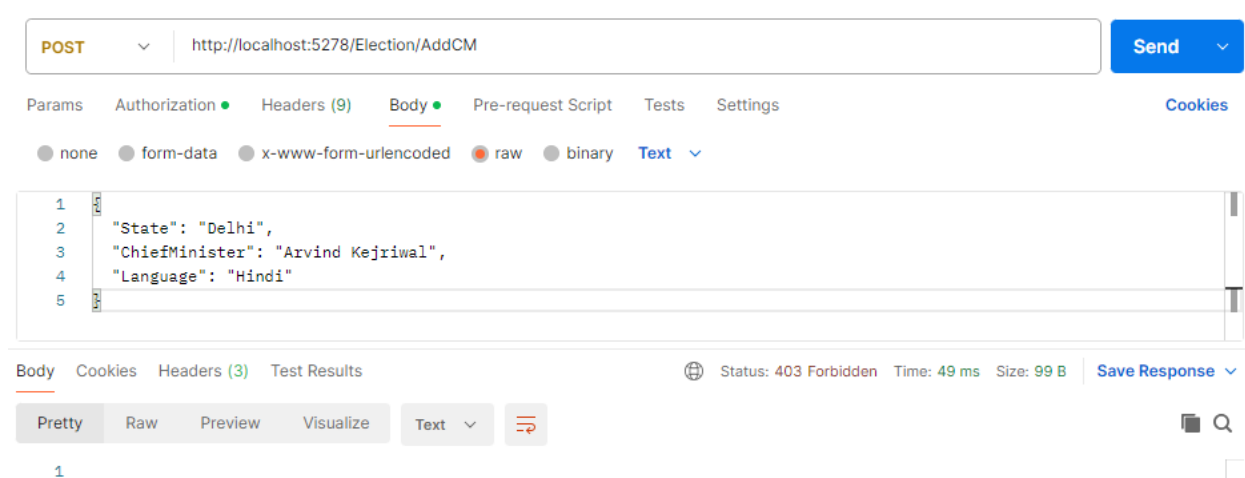
Type  Token eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

Body Cookies Headers (4) Test Results  Status: 200 OK Time: 151 ms Size: 634 B 

Pretty Raw Preview Visualize JSON  

```
1 [
2   {
3     "id": 1,
4     "state": "Kerala",
5     "chiefMinister": "Pinarayi Vijayan",
6     "language": "Malayalam"
7   },
8   {
9     "id": 2,
10    "state": "Tamil Nadu",
11    "chiefMinister": "M. K. Stalin",
12    "language": "Tamil"
13  },
14  {
15    "id": 3,
16    "state": "Karnataka",
17    "chiefMinister": "Siddaramaiah",
18    "language": "Kannada"
19  }
20 ]
```

Adding values by officer user token - Forbidden



POST  http://localhost:5278/Election/AddCM 

Params Authorization  Headers (9) Body  Pre-request Script Tests Settings Cookies

 none  form-data  x-www-form-urlencoded  raw  binary Text 

```
1 {
2   "State": "Delhi",
3   "ChiefMinister": "Arvind Kejriwal",
4   "Language": "Hindi"
5 }
```

Body Cookies Headers (3) Test Results  Status: 403 Forbidden Time: 49 ms Size: 99 B 

Pretty Raw Preview Visualize Text  

1